

ENGENHARIA DE SOFTWARE II

Prof. Júlio Machado

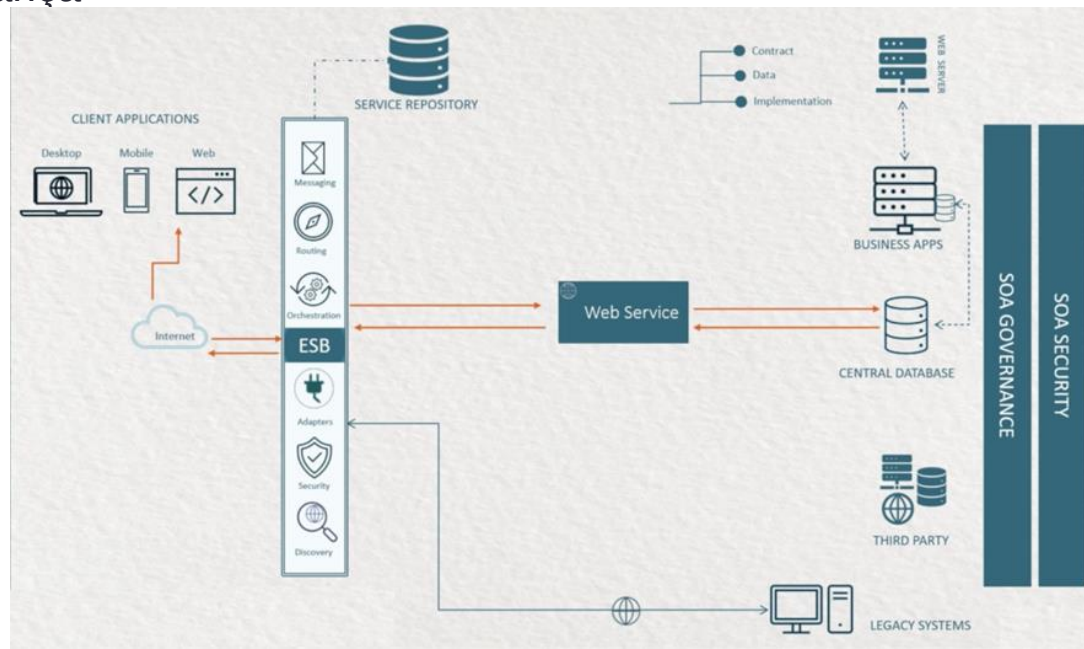
ARQUITETURA ORIENTADA A SERVIÇOS

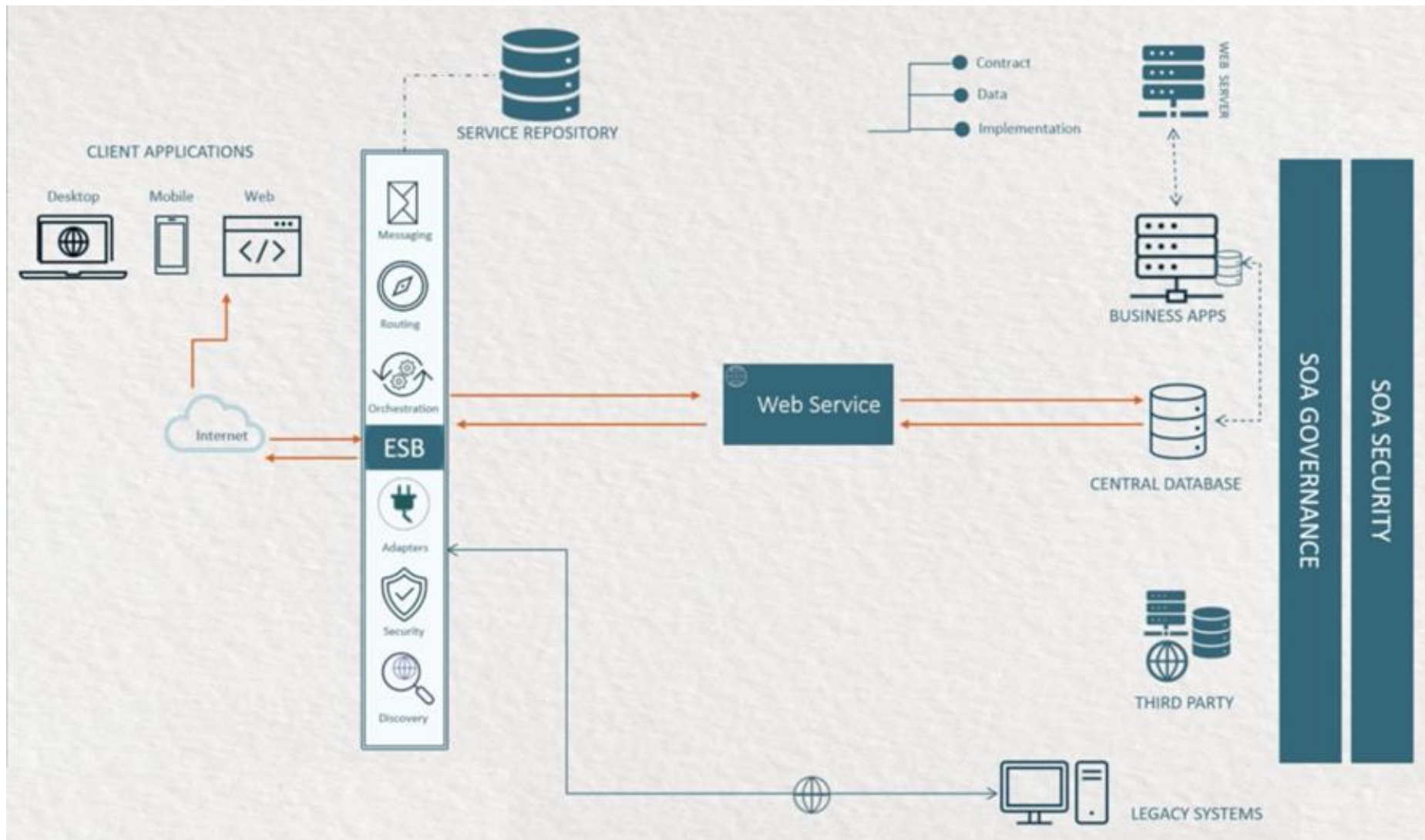
Arquitetura orientada a serviços

- Em uma arquitetura baseada em serviços (SOA) componentes e aplicações comunicam-se entre si usando protocolos bem definidos
- Arquiteturas SOA compreendem 5 elementos:
 - Serviços
 - “Service Bus” (ESB)
 - Catálogo de serviços
 - Segurança
 - Governança

Arquitetura orientada a serviços

- Requisições são enviadas usando um protocolo padrão
- Requisições são tratadas pelo ESB → orquestração e roteamento
- O ESB usa o catálogo de serviços para fazer o roteamento
- As requisições/respostas tem de estar de acordo com as regras de segurança e governança





Arquitetura orientada a serviços

- Um *serviço* é um pedaço de funcionalidade corporativa independente
 - A funcionalidade pode ser simples ou complexa
 - A idéia é que as interfaces externas dos sistemas devem ser projetadas de forma que as pessoas de negócio possam entendê-las
- A interface e o contrato de um serviço devem ser bem definidos

Arquitetura orientada a serviços

- Um *fornecedor* é um sistema que implementa um serviço de tal forma que outros sistemas podem consumi-lo
- Um *consumidor* é um sistema que utiliza um serviço fornecido

Arquitetura orientada a serviços

- Um *barramento corporativo de serviços* (ESB – Enterprise Service Bus) é a infraestrutura que possibilita a interoperabilidade entre os sistemas distribuídos para serviços
 - Torna mais fácil distribuir processos corporativos através de múltiplos sistemas utilizando diferentes plataformas e tecnologias
 - Permite a chamada de serviços entre os sistemas heterogêneos
 - Permite a implementação de controles de fluxos de trabalho

Arquitetura orientada a serviços

- Um ESB pode envolver as seguintes tarefas:
 - Prover conectividade
 - Transformar dados
 - Roteamento inteligente
 - Lidar com segurança
 - Lidar com confiabilidade
 - Gerenciamento de serviços
 - Monitoramento e logging

Lembrem-se!

- Muitas definições de SOA incluem o termo Web Service
- SOA é um conceito arquitetural
- Web Services são uma maneira possível para fornecer a infraestrutura de serviços com o uso de uma estratégia de implementação específica baseada em protocolos da Web

WEB SERVICES

Definição

- Ian Sommerville, Engenharia de Software, 8ed.:
 - “Um Web Service é um serviço, ou seja, um componente de software independente e fracamente acoplado que engloba funcionalidade discreta que pode ser distribuída e acessada por meio de uma aplicação, através de protocolos padrão.”

Definição

- W3C:
 - “Um Web Service é um sistema de software cujo propósito é suportar de maneira interoperável a interação máquina-máquina sobre uma rede de comunicação. Ele possui uma interface descrita em um formato processável por máquinas. Outros sistemas interagem com ele de acordo com a interface através de mensagens, tipicamente sobre um protocolo padrão da internet via serialização em conjunto com outros padrões web relacionados.”

Definição

- Alonso et al, Web Services – concepts, architectures and applications:
 - “A way to expose the functionality of an information system and make it available through standards Web technologies.”

Web Services

- Características:
 - Objetos remotos
 - Residem em um servidor Web e têm um endereço URL
 - Trabalham sobre o modelo de requisição/resposta
 - Utilizam protocolos que facilitam a comunicação entre sistemas
 - Independente do sistema operacional e da linguagem de programação (web services interoperáveis)
 - São objetos para soluções fracamente acopladas

Resumindo...

- Um web service é um componente de software que pode ser acessado de computadores remotos pela Internet. Dada uma entrada, um serviço produz uma saída correspondente, sem efeitos colaterais.
 - O serviço é acessado por meio de sua interface publicada e todos os detalhes da implementação do serviço ficam ocultos.
 - Os serviços não mantêm nenhum estado interno. As informações de estado são armazenadas em um banco de dados ou mantidas pelo solicitante do serviço.
- Quando uma solicitação de serviço é feita, as informações de estado podem ser incluídas como parte da solicitação e as informações de estado atualizadas são retornadas como parte do resultado do serviço.
- Como não há estado local, os serviços podem ser realocados dinamicamente de um servidor virtual para outro e replicados em vários servidores.

WEB E HTTP

Web

URI

`http://weather.example.com/oaxaca`

Identifies

Resource

Oaxaca Weather Report

Represents

Representation

Metadata:
Content-type:
`application/xhtml+xml`

Data:
`<!DOCTYPE html PUBLIC "...
 "http://www.w3.org/...
<html xmlns="http://www...
<head>
<title>5 Day Forecaste for
Oaxaca</title>
...
</html>`

HTTP

- Comunicar-se com servidores e aplicativos web se dá através do protocolo *Hypertext Transfer Protocol*
 - Protocolo de nível de aplicação
 - Protocolo textual
 - Protocolo baseado em mensagens de requisição/resposta no modelo cliente/servidor
 - Protocolo sem manutenção de estado
- Versões (em uso):
 - 1.1 (RFCs [7230](#), [7231](#), [7232](#), [7233](#), [7234](#), [7235](#))
 - 2 (RFC [7540](#))
 - ...

HTTP

- Uma requisição HTTP consiste de:
 - Uma linha inicial
 - Um ou mais campos de cabeçalho
 - Uma linha em branco
 - Possivelmente um corpo da mensagem
- Uma resposta HTTP consiste de:
 - Uma linha de status com seu código (ver [RFC](#), [Wikipédia](#)) e mensagem associada
 - Um ou mais campos de cabeçalho
 - Uma linha em branco
 - Possivelmente um corpo da mensagem

HTTP

Método de Requisição	Propósito
GET	Requisitar a transferência de representação do estado de um determinado recurso sem outros efeitos.
POST	Requisitar que um determinado recurso processe uma representação sendo enviada junto à requisição. É utilizado para realizar algum tipo de processamento com efeitos colaterais no servidor.
PUT	Requisitar que um determinado recurso crie ou altere seu estado de acordo com a representação sendo enviada junto à requisição.
PATCH	Requisitar que um determinado recurso altere seu estado de acordo com a representação parcial sendo enviada junto à requisição. Difere do PUT pois permite uma alteração parcial, enquanto PUT executa uma alteração total.
DELETE	Requisitar que um determinado recurso tenha seu estado removido.

HTTP

Método	RFC	Requisição com corpo	Resposta com corpo	Seguro	Idempotente	Suporta cache
GET	RFC 9110	Opcional	Sim	Sim	Sim	Sim
HEAD	RFC 9110	Opcional	Não	Sim	Sim	Sim
POST	RFC 9110	Sim	Sim	Não	Não	Sim
PUT	RFC 9110	Sim	Sim	Não	Sim	Não
DELETE	RFC 9110	Opcional	Sim	Não	Sim	Não
CONNECT	RFC 9110	Opcional	Sim	Não	Não	Não
OPTIONS	RFC 9110	Opcional	Sim	Sim	Sim	Não
TRACE	RFC 9110	Não	Sim	Sim	Sim	Não
PATCH	RFC 5789	Sim	Sim	Não	Não	Não

https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol

HTTP

- Propriedades dos métodos de requisição:
 - Seguro: um método de requisição é seguro se não realiza efeitos colaterais (tais como alterações de dados) no servidor.
 - Idempotente: um método de requisição é idempotente quando múltiplas requisições idênticas tem o mesmo efeito.
 - Suporte a cache: um método de requisição tem suporte a cache quando a respectiva resposta pode ser armazenada para reuso futuro.

HTTP

- Protocolo HTTP possui uma coleção de códigos de status para as mensagens de resposta:

Grupo de códigos	Propósito
1xx	Requisição recebida, compreendida e o processamento continua.
2xx	Requisição recebida, compreendida e processada com sucesso.
3xx	Ação adicional é necessária para continuar o processamento da requisição.
4xx	Requisição possui alguma falha com origem no lado cliente ou não pode ser processada.
5xx	Servidor não conseguiu processar uma requisição aparentemente válida.

HTTP

- Veja mais detalhes sobre códigos de status em:
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
 - <https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml>

WEB SERVICES

REST

REST

- REST é um acrônimo para *Representational State Transfer*
 - Dissertação de doutorado de Roy Fielding “Architectural Styles and the Design of Network-based Software Architectures”, disponível em <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

REST

- REST é um estilo arquitetural para aplicações Web baseado em quatro princípios:
 1. Identificação dos recursos através de URIs;
 2. Interface uniforme de acesso aos recursos via métodos de requisição do protocolo HTTP;
 3. Mensagens auto descritivas;
 4. Interação através de hiperlinks.

REST

- Exemplos:
 - <http://jsonplaceholder.typicode.com/>
 - <https://randomuser.me/>
 - <https://reqres.in/>
 - <http://petstore.swagger.io/>

REST

- Tarefas para o desenvolvedor:
 - Definir quais são os recursos expostos
 - Definir o formato das URIs para os recursos
 - Decidir quais métodos do HTTP serão utilizados
 - Estabelecer a real semântica da aplicação de cada método sobre um recurso

REST

Método	Escopo	Semântica CRUD
GET	Coleção	Obter uma coleção de recursos. READ
GET	Recurso	Obter um recurso. READ
POST	Coleção	Criar um novo recurso em uma coleção. CREATE
PUT	Recurso	Atualizar totalmente um recurso. UPDATE
PATCH	Recurso	Atualizar parcialmente um recurso. UPDATE
DELETE	Recurso	Remover um recurso. DELETE

REST

- Exemplo: URI de coleção de recursos
 - `http://exemplo/recursos/`
 - GET: lista as URIs e outros detalhes dos elementos da coleção
 - PUT: substitui a coleção por uma outra
 - PATCH: não é muito utilizado
 - POST: adiciona um novo elemento na coleção, retornando a URI para o novo elemento
 - DELETE: remove a coleção inteira

REST

- Exemplo: URI de recurso individual
 - `http://exemplo/recursos/123`
 - GET: obtém a representação de um elemento específico da coleção
 - PUT: substitui um membro específico da coleção ou, se ele não existe, cria um novo
 - PATCH: atualiza (podendo ser só uma parte) do membro específico da coleção
 - POST: geralmente não utilizado; trata o elemento da coleção como uma própria coleção, adicionando um novo elemento nele
 - DELETE: remove o elemento da coleção

REST

- Serialização de informação em vários formatos:
 - XML
 - JSON – JavaScript Object Notation
 - Texto
 - etc

JSON

- JSON = JavaScript Object Notation
- Formato textual para serialização de dados
- Documentação: <http://json.org/>

```
{  
  "productName": "Computer Monitor",  
  "price": "229.00",  
  "specifications": {  
    "size": 22,  
    "type": "LCD",  
    "colors": ["black", "red", "white"]  
  }  
}
```

OData

- Open Data Protocol
- Padrão OASIS para interfaces REST
- Documentação: <http://www.odata.org/>

Documentação de APIs

- APIs de web services REST serão consumidos pelos desenvolvedores do lado cliente de uma aplicação
- Nem sempre serão os mesmos desenvolvedores envolvidos na implementação do backend
- É essencial uma boa documentação das funcionalidades expostas pela API REST

Documentação de APIs

- Dois modelos de trabalho:

“Design first”

- Projetar e documentar a API antes da codificação do web service.

“Code first”

- Gerar a documentação a partir do código-fonte do web service.

Documentação de APIs

- A especificação OpenAPI é uma linguagem de especificação para APIs baseadas em HTTP
 - <https://www.openapis.org/>

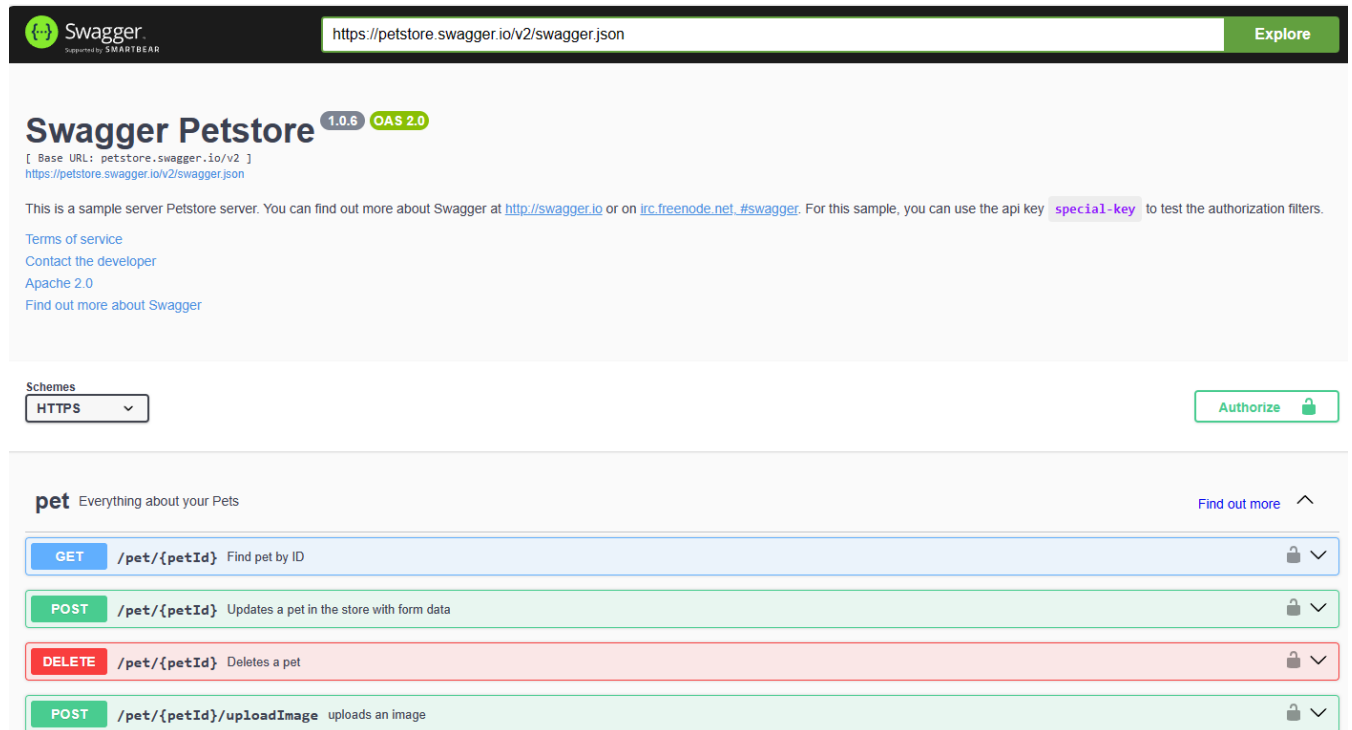


- Swagger é um toolkit de ferramentas que implementam e suportam OpenAPI
 - <https://swagger.io/>



Swagger

- Exemplo:
- <https://petstore.swagger.io/>
- <https://petstore3.swagger.io/>



The screenshot displays the Swagger Petstore API interface. At the top, the Swagger logo is on the left, and a search bar contains the URL `https://petstore.swagger.io/v2/swagger.json` with an **Explore** button. Below this, the title **Swagger Petstore** is shown with version tags **1.0.6** and **OAS 2.0**. A note indicates the base URL is `petstore.swagger.io/v2` and provides a link to the Swagger JSON file. A descriptive paragraph states this is a sample server and provides links for terms of service, developer contact, Apache 2.0 license, and more about Swagger. An **Authorize** button is present. Under the **Schemes** dropdown (set to **HTTPS**), a list of API endpoints is shown under the **pet** namespace, described as 'Everything about your Pets'. The endpoints include: **GET** `/pet/{petId}` (Find pet by ID), **POST** `/pet/{petId}` (Updates a pet in the store with form data), **DELETE** `/pet/{petId}` (Deletes a pet), and **POST** `/pet/{petId}/uploadImage` (uploads an image). Each endpoint entry includes a method label, the path, a brief description, and a lock icon.

Swagger
powered by SMARTBEAR

`https://petstore.swagger.io/v2/swagger.json` **Explore**

Swagger Petstore **1.0.6** **OAS 2.0**

[Base URL: `petstore.swagger.io/v2`]
<https://petstore.swagger.io/v2/swagger.json>

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on [#swagger">irc.freenode.net #swagger](irc.freenode.net). For this sample, you can use the api key `special-key` to test the authorization filters.

[Terms of service](#)
[Contact the developer](#)
[Apache 2.0](#)
[Find out more about Swagger](#)

Schemes
HTTPS **Authorize**

pet Everything about your Pets [Find out more](#)

GET	<code>/pet/{petId}</code>	Find pet by ID	🔒
POST	<code>/pet/{petId}</code>	Updates a pet in the store with form data	🔒
DELETE	<code>/pet/{petId}</code>	Deletes a pet	🔒
POST	<code>/pet/{petId}/uploadImage</code>	uploads an image	🔒

WEB SERVICES

Outros

Outro Modelos de Serviços Web

- GraphQL
 - <https://graphql.org/>
 - Interface de serviços baseada em uma linguagem de consulta
- gRPC
 - <https://grpc.io/>
 - Interface de serviços baseada em chamada remota de procedimentos